

Set 2

1. Q1. Write a Python function `is\_prime(n)` to check if a number is prime. Use an AI coding tool to generate and justify a set of test cases before implementation. Pass all generated test cases.

(Each question carries 5 marks)

Prompt:

Generate justified test cases for a Python function `is_prime(n)` covering normal, edge, negative, and boundary values. Then write an efficient `is_prime(n)` function that passes all those test cases.

Code:

```
▶ passed_tests = 0
   failed_tests = 0

   print("\n--- Running Test Cases ---")
   for test_case in test_cases:
       num = test_case["input"]
       expected = test_case["expected"]
       explanation = test_case["explanation"]

       actual = is_prime(num)

       status = "PASSED" if actual == expected else "FAILED"
       if status == "PASSED":
           passed_tests += 1
       else:
           failed_tests += 1

       print(f"Input: {num:<5}, Expected: {expected:<5}, Actual: {actual:<5} -> {status} ({explanation})")

   print("\n--- Test Summary ---")
   print(f"Total Tests Run: {len(test_cases)}")
   print(f"Tests Passed: {passed_tests}")
   print(f"Tests Failed: {failed_tests}")

   if failed_tests == 0:
       print("All test cases passed! The `is_prime` function appears to be correct.")
   else:
       print(f"Some test cases failed. Please review the `is_prime` function and test cases.")
```

Output:

```
*** --- Running Test Cases ---
Input: 2 , Expected: 1 , Actual: 1 -> PASSED (Smallest and only even prime number.)
Input: 3 , Expected: 1 , Actual: 1 -> PASSED (Typical small prime number.)
Input: 5 , Expected: 1 , Actual: 1 -> PASSED (Typical small prime number ending in 5.)
Input: 7 , Expected: 1 , Actual: 1 -> PASSED (Typical small prime number.)
Input: 11 , Expected: 1 , Actual: 1 -> PASSED (Typical prime number, double digit.)
Input: 4 , Expected: 0 , Actual: 0 -> PASSED (Smallest composite number (2*2).)
Input: 6 , Expected: 0 , Actual: 0 -> PASSED (Composite number (2*3).)
Input: 8 , Expected: 0 , Actual: 0 -> PASSED (Composite number (2*2*2).)
Input: 9 , Expected: 0 , Actual: 0 -> PASSED (Composite number (3*3).)
Input: 10 , Expected: 0 , Actual: 0 -> PASSED (Composite number (2*5).)
Input: 0 , Expected: 0 , Actual: 0 -> PASSED (Edge case: 0 is not prime by definition.)
Input: 1 , Expected: 0 , Actual: 0 -> PASSED (Edge case: 1 is not prime by definition.)
Input: -1 , Expected: 0 , Actual: 0 -> PASSED (Edge case: Negative numbers are not prime.)
Input: -5 , Expected: 0 , Actual: 0 -> PASSED (Edge case: Negative numbers are not prime.)
Input: 2 , Expected: 1 , Actual: 1 -> PASSED (Specifically testing the smallest prime number,
Input: 12 , Expected: 0 , Actual: 0 -> PASSED (Multiple of 2 and 3 (2*2*3).)
Input: 15 , Expected: 0 , Actual: 0 -> PASSED (Multiple of 3 and 5 (3*5).)
Input: 25 , Expected: 0 , Actual: 0 -> PASSED (Multiple of 5 (5*5).)
Input: 97 , Expected: 1 , Actual: 1 -> PASSED (Larger prime number to test efficiency and corre
Input: 100 , Expected: 0 , Actual: 0 -> PASSED (Larger composite number (10*10) to test efficien
Input: 997 , Expected: 1 , Actual: 1 -> PASSED (Large prime number.)
Input: 1001 , Expected: 0 , Actual: 0 -> PASSED (Large composite number (7*11*13).)

--- Test Summary ---
Total Tests Run: 22
Tests Passed: 22
Tests Failed: 0
All test cases passed! The `is_prime` function appears to be correct.
```

2. Q2. Develop a Python class `Rectangle` with methods for area and perimeter. Use an AI assistant for auto-generating docstrings and inline comments. Compare them with your manual documentation.

(Each question carries 5 marks)

Prompt:

Create a Python class Rectangle with methods for area and perimeter. Auto-generate docstrings and inline comments using an AI assistant. Then provide my own manual documentation so I can compare both.

Code:

```
▶ # Demonstrate the Rectangle class

# Create a rectangle object
rect1 = Rectangle(width=5, height=10)

# Calculate and print the area
area1 = rect1.area()
print(f"Rectangle 1: Width = {rect1.width}, Height = {rect1.height}")
print(f"Area: {area1}")

# Calculate and print the perimeter
perimeter1 = rect1.perimeter()
print(f"Perimeter: {perimeter1}")

print("\n---")

# Create another rectangle object with different dimensions
rect2 = Rectangle(width=7.5, height=3.2)

# Calculate and print the area
area2 = rect2.area()
print(f"Rectangle 2: Width = {rect2.width}, Height = {rect2.height}")
print(f"Area: {area2}")

# Calculate and print the perimeter
perimeter2 = rect2.perimeter()
print(f"Perimeter: {perimeter2}")

▶ print("\n--- Testing edge cases (invalid input) ---")
try:
    invalid_rect = Rectangle(width=-1, height=5)
except ValueError as e:
    print(f"Error creating rectangle with invalid dimensions: {e}")
```

Output:

```
... Rectangle 1: Width = 5, Height = 10
Area: 50
Perimeter: 30

---
Rectangle 2: Width = 7.5, Height = 3.2
Area: 24.0
Perimeter: 21.4

--- Testing edge cases (invalid input) ---
Error creating rectangle with invalid dimensions: Width and height must be positive values.
```